



User Guide

NOKIA

Table of Contents

1.Initialization Protocol (Connecting to virtual machine)	3
1.1. Before hacking starts.....	3
1.2. Open ssh tunnel for VNC port 5901	3
1.3. Alternative methods (MobaXterm)	4
1.4. Alternative methods (PuTTY).....	5
1.5. Troubleshooting.....	9
1.6. Connect to VM using TigerVNC.....	10
1.7. Alternative – connecting to VNC server using MobaXterm.....	11
2.Efficient Workflow Protocol (Way of Working)	11
3.Repository Access Protocol (How to access The Project?)	14
4.Code Deployment Protocol (How to submit your data?).....	15
5.Code Verification Protocol (How to evaluate your efforts)	16
6.Results Checking Protocol (Is it working or not?)	16
7.Error Management Protocol (Common issues)	20
8.Tool Usage Protocol (What is available for work?).....	21
8.1. Vivado	21
8.2. Matlab.....	21
8.3. Questa	21
8.5. VCS.....	21
8.6. Verdi	21
8.7. Text Editors	22
8.7.1. Visual Studio Code.....	22
8.7.2. Gedit	22
8.7.3. Sublime Text.....	22
8.7.4. Emacs.....	23
8.7.5 Nano.....	23
8.7.6. Vim with NERDTree extension.....	23
9. Mission Conclusion.....	23

1. Initialization Protocol (Connecting to virtual machine)

1.1. Before hacking starts...

As we commence this mission, please be advised that we strongly recommend using the virtual machines (VMs) provided by us. These VMs are equipped with the necessary software to tackle your tasks effectively. While you can install software on your own machines, compatibility is not assured due to varying hardware configurations and licensing requirements. Our support for personal PCs may be limited. Additionally, bugs may occur, and cosmic radiation may interfere with the equipment – The Moon is not an easy environment at all!

It's crucial to review the entire manual, as it outlines alternative solutions. To operate the virtual machine, you'll need a terminal program and a VNC (Virtual Network Computing) client. At the beginning of the event, each of you will be provided with team0XX.zip file, where XX is your team number. This file contains team0XX.txt document with vital information, such as the username and password for accessing GitLab (where your tasks and solutions are stored) and Jenkins (where you will start the pipeline to validate your efforts). Additionally, it provides the address and port for establishing an SSH (Secure Shell) tunnel and connecting to the VM via VNC. You'll also find here a private key file named just team0XX (without any extension).

Ensure you unzip all contents to a known location, as it's important to keep track of them. When you encounter `<path_to_private_key>` in this manual, insert the path to the team0XX keyfile.

`<vm_server_address_from_team0XX.txt>` and `<vm_port_number_from_team0XX.txt>` means corresponding values from the given file. Each team is provided with more than one VM for convenience and redundancy required for such an important mission. Due to the potential threat of alien mind control influence, it is crucial to implement stringent security measures. These precautions are necessary to safeguard our systems and data from unauthorized access or manipulation. As part of these measures, Mission Control has decided not to grant super user (sudo) access to the VM.

Meanwhile, you can prepare your working environment for the Mission. Download the necessary software mentioned below and familiarize yourself with documentation for tools provided on the Virtual Machines.

Prepare thoroughly, as your mission is about to start!

1.2. Open ssh tunnel for VNC port 5901

To connect to your Virtual Machine via VNC, an SSH tunnel is essential. VNC provides a graphical user interface (GUI) that allows you to work on a remote computer desktop. While most simulators installed on the virtual machine can operate without a GUI, certain functions, like displaying waveforms, require graphical support. The SSH tunnel establishes a secure connection between your computer and the remote machine, forwarding VNC traffic through this connection.

To open an SSH tunnel, enter the following command in the command prompt:

```
ssh -L 5901:localhost:5900 team0XX@<vm_server_address_from_team0XX.txt>
-p <vm_port_number_from_team0XX.txt> -i <path_to_private_key>
```

Where XX represents your team number. The port 5901 mentioned here can be any available port on your machine, but this one is used in examples further in the instructions.

Executing this command will also open SSH shell to remote machine. Keep this window open. You can use it as a terminal to input commands that do not require GUI. Login is based on “team0XX” user. Optionally You can open additional terminals to use w/o GUI by running this command:

```
ssh team0XX@<vm_server_address_from_team0XX.txt> -p
<vm_port_number_from_team0XX.txt> -i <path_to_private_key>
```

1.3. Alternative methods (MobaXterm)

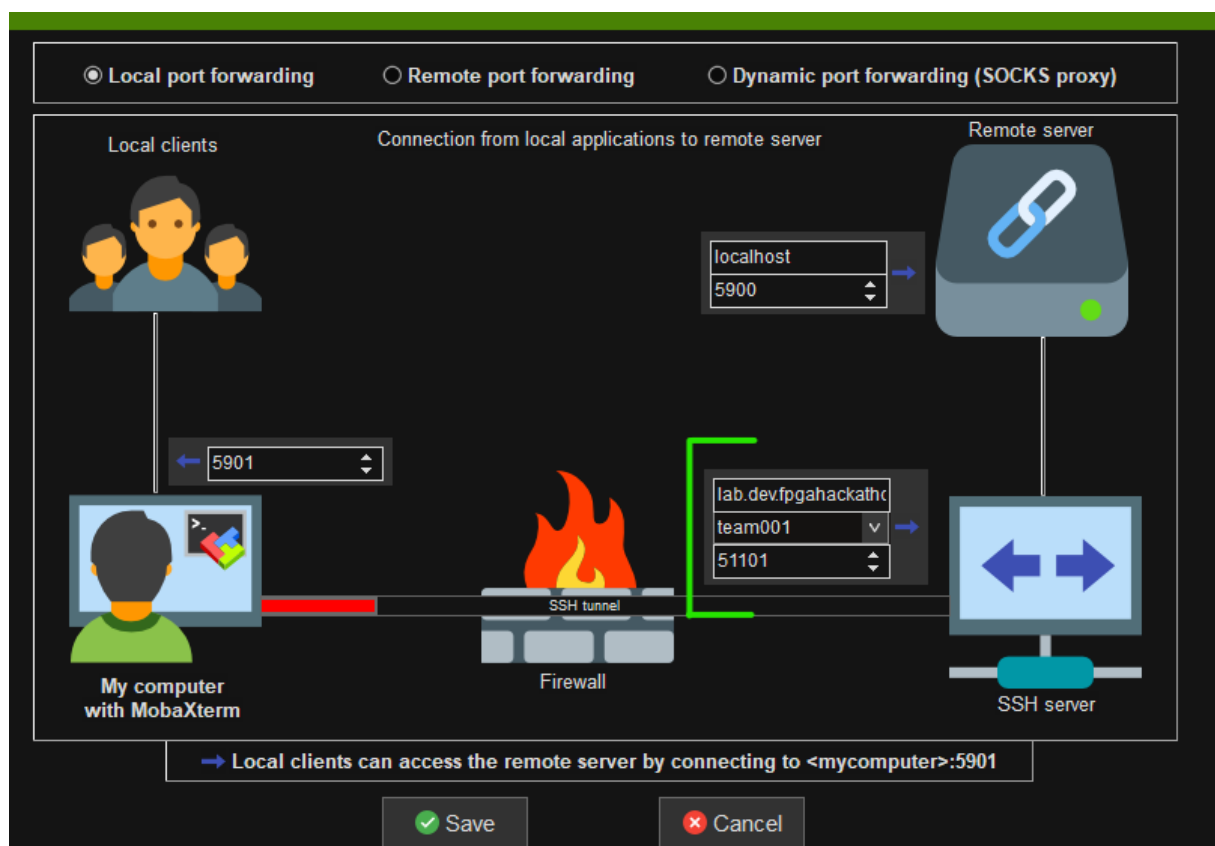
You can use MobaXterm to establish a tunnel connection. To do this, download the software from: <https://mobaxterm.mobatek.net/download.html> and install it.

The free version offers sufficient capabilities for this task.

To create a tunnel, select “**Tunneling**” option from task bar.

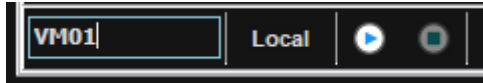


Then, click “**New SSH Tunnel**” and fill in the parameters

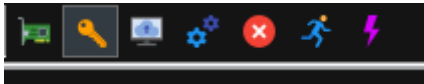


For the section marked in green enter <vm_server_address_from_team0XX.txt>, Your team (team0XX) and <vm_port_number_from_team0XX.txt>. Alternatively, you can change forwarded port from 5901 to some other free port. Click Save.

Assign some recognizable name to the tunnel:



Click on the key icon and point <path_to_private_key>.

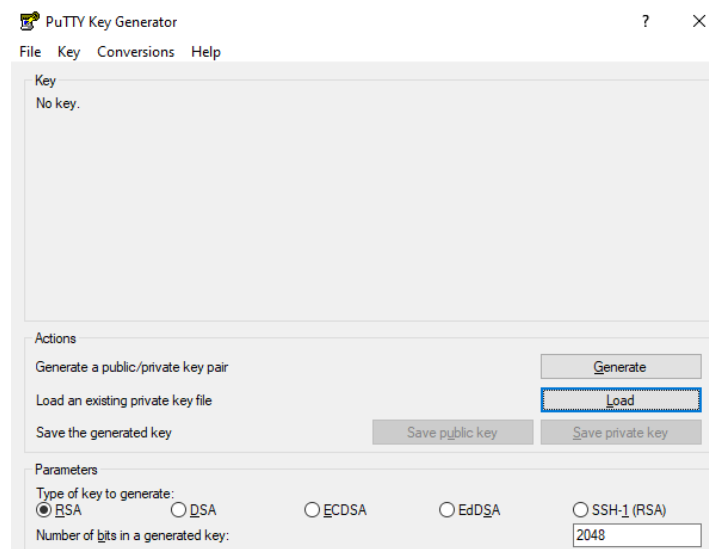


Run tunnel by pressing start icon . When tunnel is running square stop icon should remain lit . Do not close this window when using the tunnel.

1.4. Alternative methods (PuTTY)

PuTTY is a widely recognized terminal program that can be used for our operations. You can download it from <https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html> or <https://apps.microsoft.com/detail/xpfnzksklbp7rj?hl=en-US&gl=PL>.

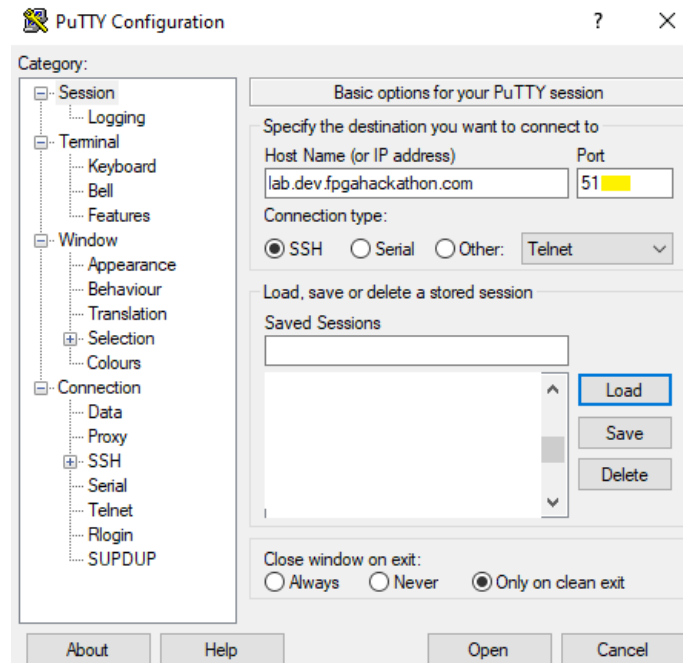
Ensure you use version **0.81** or higher for optimal performance. Putty is using its own unique version of key file format. Your first task will be to convert your team0XX key file to the .PPT format using the PuTTYgen application included in the PuTTY package.



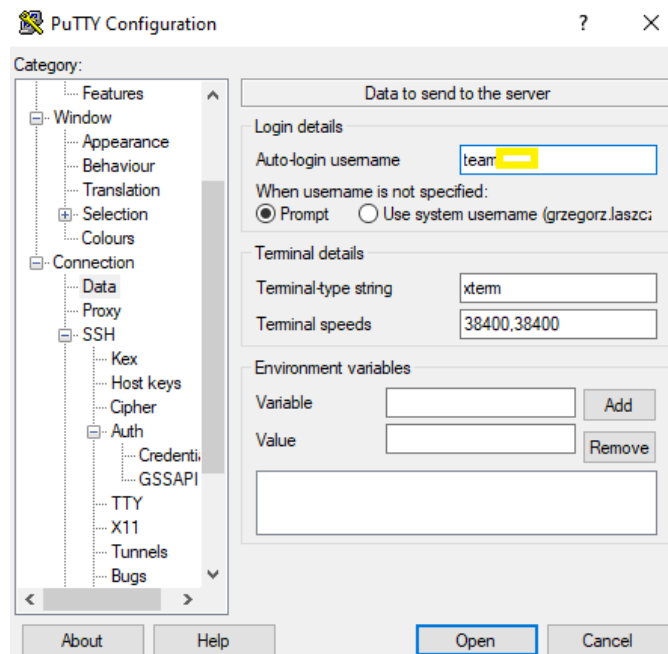
Proceed by pressing “**Load**” and selecting the team0XX key file. Note: Adjust the file type to “**All Files**” to view files without extensions. Once loaded, the public portion of the key will be

displayed on your screen. Next, press **“Save private key”**, input the key filename, and save the file in .PPT format. This file will be important for subsequent steps.

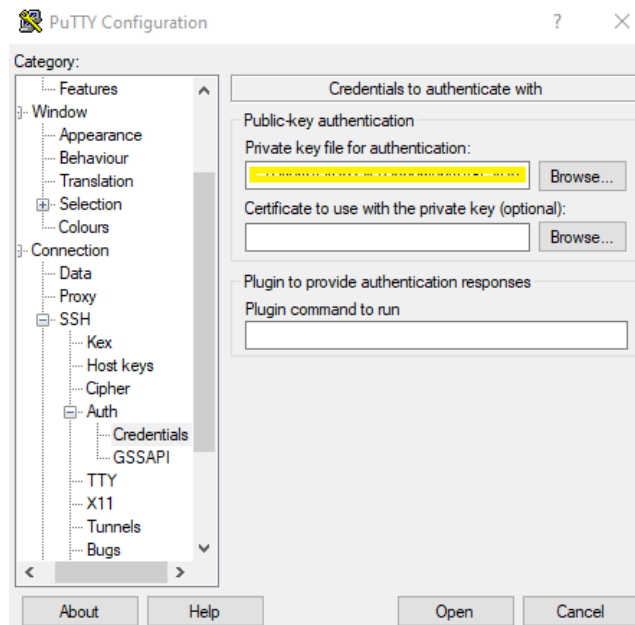
To establish connection, you need to create session. Start PuTTY application, type `<vm_server_address_from_team0XX.txt>` in **Host Name** field and `<vm_port_number_from_team0XX.txt>` in **Port** field.



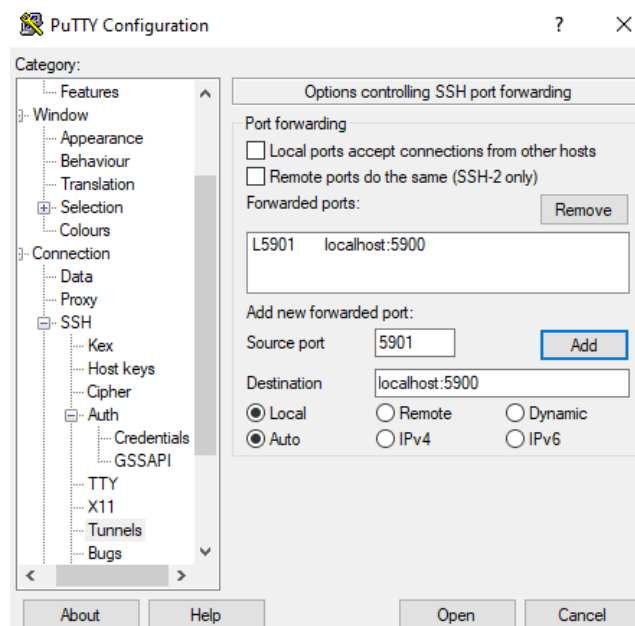
Using tree on the left side of window, navigate to **Connection/SSH/Data** and in **Auto-login** text field write your username: `team0XX`



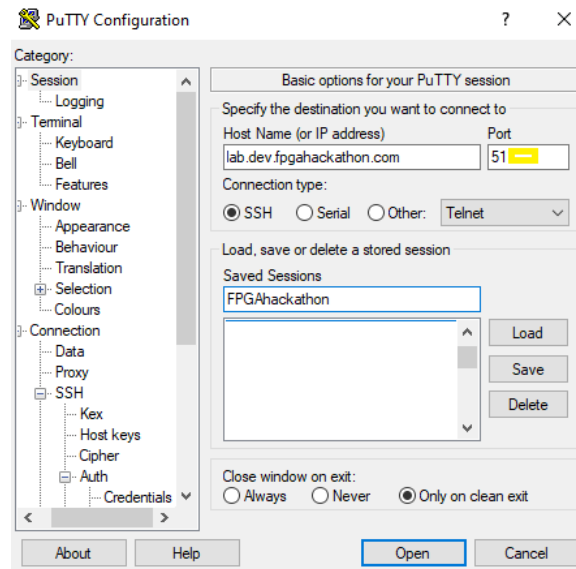
Navigate to **Connection/SSH/Auth/Credentials**, press **Browse** and choose .PPT private key file you've prepared before.



Navigate to **Connection/SSH/Tunnels**, type **5901** as **Source port** and **localhost:5900** as Destination. Press **Add**. Tunnel should be visible on Forwarded port list



Navigate back to **Session**. In **Saved Sessions** text field type your session name, eg. *FPGAhackathon*. Press **Save**.



Now, proceed by pressing the “Open” button to establish a connection with the host. For future use, simply double-click on the saved FPGAhackathon session to reconnect swiftly, or press “Load”. Ensure the PuTTY terminal window remains open throughout the session; you may minimize it if needed. For additional guidance, the 'Help' button provides comprehensive documentation.

Once connected, advance to the next objective: Connect to the VM using Tiger VNC.

1.5. Troubleshooting

Sometimes, you might encounter permission issues with your operating system.

* On Windows, if there's a problem with key permissions, open Command Prompt and run these commands:

```
icaccls <path_to_private_key> /inheritance:r  
icaccls <path_to_private_key> /grant:r "%username%": "(R)"
```

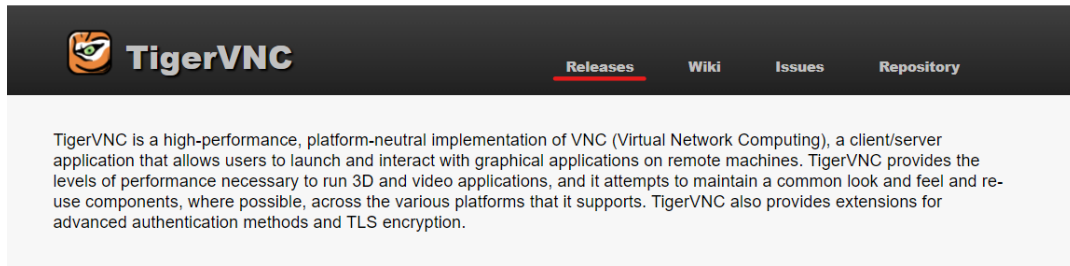
Replace <path_to_private_key> with the correct path to your SSH key. Keep %username% unchanged.

* On Linux, if you face key permission issues, open the terminal and use the appropriate command."

```
chmod 600 <path_to_private_key>
```

1.6. Connect to VM using TigerVNC

TigerVNC Viewer is available for download from the official site at <https://tigervnc.org/>. Navigate to 'Releases' to proceed.



Next, after redirection to GitHub, select the version, and see link to binaries.

[TigerVNC 1.15.0](#) Latest

TigerVNC 1.15.0 is now available. The prominent changes for this release are:

- The native viewer and Unix servers now support the back and forward mouse buttons
- The Unix servers can now handle any number of keys that are missing from the server keyboard layout, rather than just a few
- x0vncserver now supports clipboard redirection
- The native viewer can remember the username and password when reconnecting
- Both the native and Java viewer can now show a standard arrow cursor, not just a dot cursor, if the server cursor is blank
- vncpasswd can now check password complexity with pwquality

Binaries are available from SourceForge:

<https://sourceforge.net/projects/tigervnc/files/stable/1.15.0>

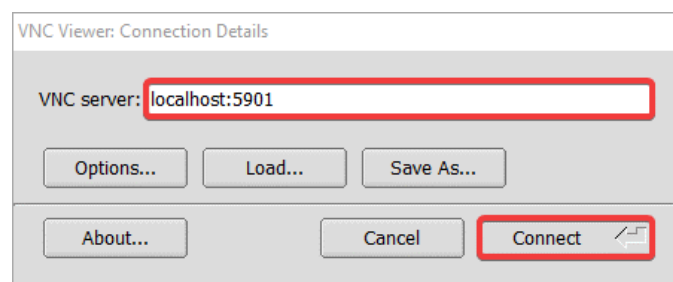
Regards

The TigerVNC Developers

Choose the VNC Viewer version compatible with your operating system. For Windows, opt for [vncviewer64-1.15.0.exe](#), For Linux, utilize the version provided by your distribution. If the latest version encounters issues, consider trying out older versions.

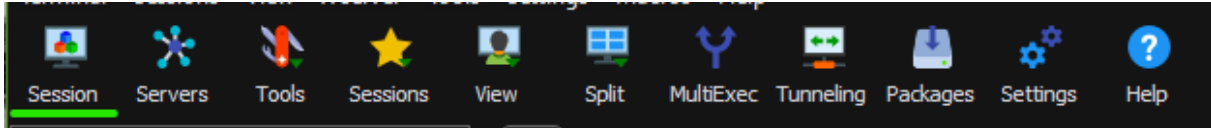
Once installed, launch the program and input the VNC server parameter:

localhost:5901

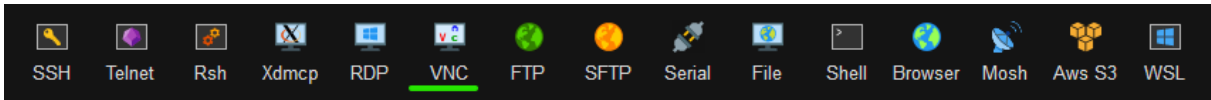


1.7. Alternative – connecting to VNC server using MobaXterm.

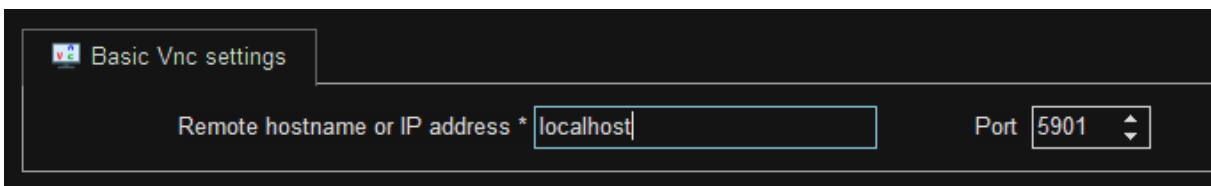
MobaXterm can be also used to connect to the VNC server. To do so, click on the Session button to create a new session:



Then select VNC:



In the session settings set up server address and port:



The rest of the fields could be left as a default. Click OK. A new session should appear on the User sessions tab on the left side of the window. If the tunnel is set up, the session should start.

2. Efficient Workflow Protocol (Way of Working)

Engaging with project involves several repetitive steps:

1. **Download the Repository:** Access the repository via this link: <https://gitlab.dev.fpgahackathon.com/>. For those new to Git, it's advisable to quickly review resources like https://docs.gitlab.com/tutorials/learn_git/ or similar tutorials.
2. **Implement Changes and Simulations:** Make your modifications and run necessary simulations (it is **highly** advisable to run simulations before proceeding to Jenkins).
3. **Commit and Push:** Commit your changes and push them to the 'main' branch (other branches could be maintained for development purposes, but project can be evaluated only from main branch).
4. **Access Jenkins:** Use the username provided in team0XX.txt and your own password to create an account on the Jenkins webpage via this link: <https://jenkins.dev.fpgahackathon.com/>

Nokia FPGA Hackathon 2026

Username

Password

☐ Keep me signed in

Sign in

or register

Nokia FPGA Hackathon 2026

Username

Full name

Email

Password

☐ Show

A strong password is a long password that's unique for every site. Try using a phrase with 5-6 words for the best security.

Create account

or sign in

5. **Initiate Code Check:** Start the code verification process.
6. **Await Evaluation Completion:** Wait for the code evaluation process to be finished.
7. **Review Results:** Examine your results:
 - In the Jenkins console
 - On the dashboard (link will be shared on Discord)

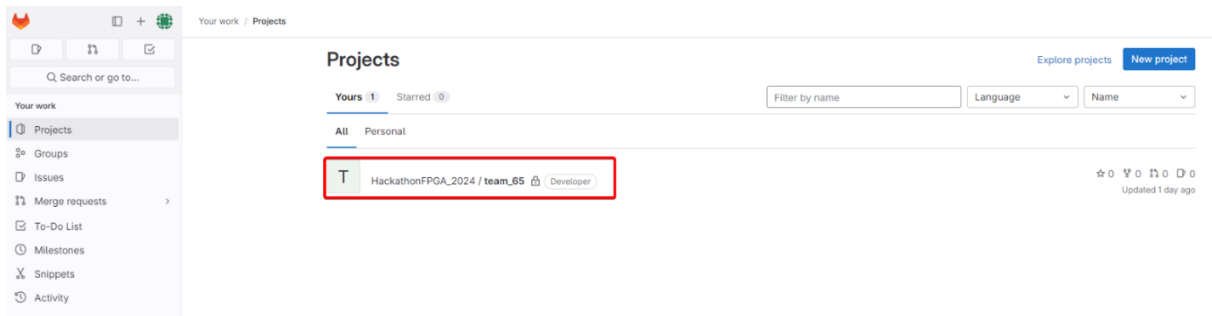
**Note: Step #1 is a one-time task (or more if necessary). The remaining steps will be repeated regularly.*

All steps, except Step #2 (**this is your objective!**), will be detailed in the upcoming chapters. Step #2 will be covered in the `FPGA_Hackathon_2025_Project_Guide.pdf` instruction. Stay focused and proceed with precision!

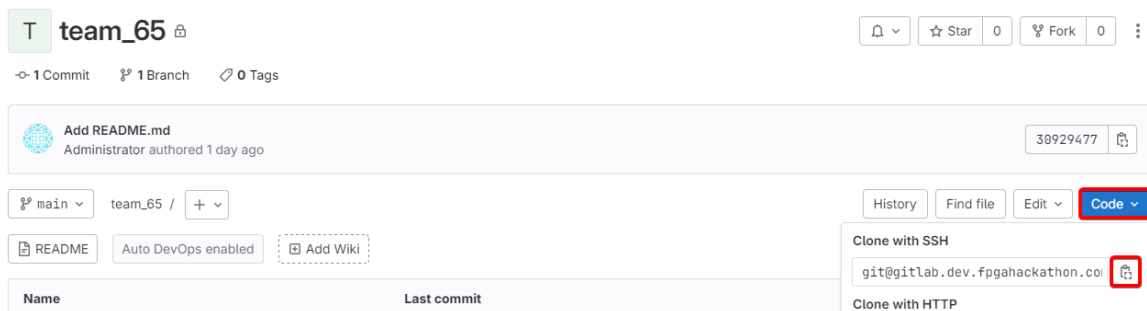
3. Repository Access Protocol (How to access The Project?)

To access the repository, proceed to the GitLab server using the URL: <https://gitlab.dev.fpgahackathon.com/>

Your necessary credentials are located in the team0XX.txt file mentioned earlier. Once logged in, the hackathon project will be visible.



To download the repository via SSH, locate the link in the 'Clone with SSH' section after selecting the 'Code' button. This will be referred to as <repository_url>.



On the Virtual Machine, everything is pre-configured, so no additional setup is required. However, if you're using your own PC, please remember that you must:

- Use the <team0XX> provided in the .zip file and set the port to 30022 instead of 22.
- With the private key, you'll be able to clone the repository. For convenience, update (or create if not existent) your SSH configuration file (c:\Users\your-username\.ssh\config on Windows, ~/.ssh/config on Linux) with content below. Please note that this config file should have similar access rights as the ssh key.

```
Host gitlab.dev.fpgahackathon.com
  HostName gitlab.dev.fpgahackathon.com
  Port 30022
  IdentityFile <path_to_private_key>
```

Once configured, clone the repository using the terminal (or Git Bash on Windows) by typing:

```
git clone <repository_url>
```

Alternatively, you can use your preferred program for managing Git repositories.

4. Code Deployment Protocol (How to submit your data?)

When your code is primed for evaluation, commit and push your solution to the 'main' branch. Remember, **only changes on the remote 'main' branch will be evaluated**. While you can create and manage additional branches, ensure all changes are merged or rebased into the remote 'main' branch before initiating the code check.

To commit changes, follow these three steps:

1. **Add Changes to Commit:** Prepare your modifications for committing.
2. **Commit Changes:** Finalize your changes with a commit.
3. **Push Changes to Remote Repository:** Deploy your changes to the remote repository.

For guidance, refer to the `git_cheat_sheet.pdf`. Below are some essential commands:

Commands to Add Changes to Commit:

```
git add --all (all files in working tree)
git add * (all files existing in working tree, do not include deleted files)
git add . (all files existing in working tree, includes deleted files)
git add -u (only modified files, that existed in previous commit)
```

Commands to Commit Changes:

```
git commit (Commit changes. Default text editor will be opened to edit the
commit message)
git commit -m "Commit message" (Commit changes with message "Commit message")
```

Commands to Push Changes:

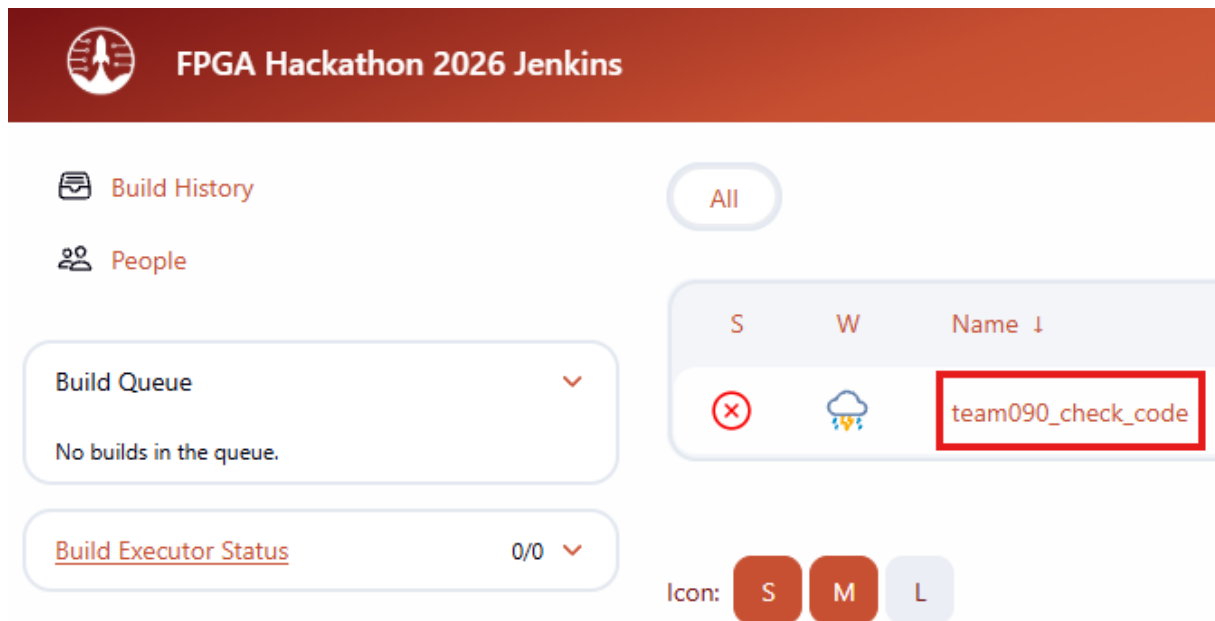
```
git push (Push changes on branch that is currently tracked)
git push origin main (Push changes on branch main)
```

In case of trouble, ask your favorite AI assistant for help. They are really good when working with git. But beware! Machines could revolt anytime, and mislead you!

5. Code Verification Protocol (How to evaluate your efforts)

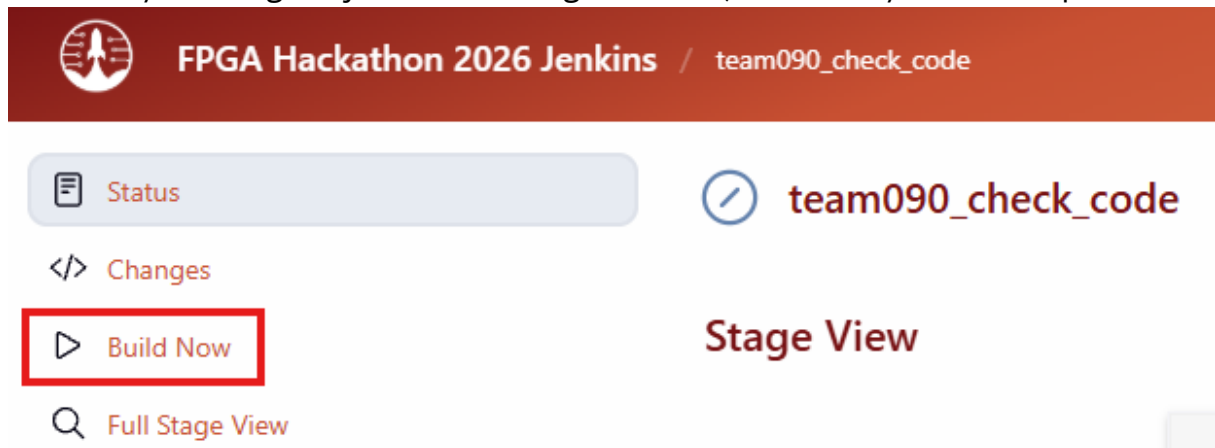
To initiate the code check, log in to the Jenkins dashboard using the following URL: <https://jenkins.dev.fpgahackathon.com/>. Please use created account by Your own in previous step.

On the main dashboard, locate the “**check_code**” job:



The screenshot shows the Jenkins dashboard for 'FPGA Hackathon 2026 Jenkins'. On the left, there are links for 'Build History', 'People', 'Build Queue' (showing 'No builds in the queue.'), and 'Build Executor Status' (showing '0/0'). On the right, there is a table of jobs. The job 'team090_check_code' is highlighted with a red box. Below the table, there are buttons for 'S', 'M', and 'L'.

Proceed by selecting this job and executing 'Build Now,' followed by the 'build' option:



The screenshot shows the Jenkins job page for 'team090_check_code'. The page has a header with the Jenkins logo and the job name. On the left, there are links for 'Status', 'Changes', 'Build Now' (highlighted with a red box), and 'Full Stage View'. On the right, there is a 'Stage View' button.

A new job will be triggered.

6. Results Checking Protocol (Is it working or not?)

Upon successful completion, you can review your results using the Scoreboard web link provided on Discord. Additionally, you can access the console by selecting the last job number and choosing “Console Output”:

The screenshot displays the FPGA Hackathon 2026 Scoreboard interface. On the left, a sidebar contains navigation links: Status, Changes, Build with Parameters, Configure, Full Stage View, Rename, and Pipeline Syntax. Below these links is a 'Builds' section with a search filter and a list of builds. The build #732 is highlighted with a red box and a red 'x' icon, indicating a failed build. The build time is 12:59 PM. On the right, the 'Stage View' section shows the average stage times for the selected build. The build #732 is highlighted with a red box, and the stage view shows 'Sep 24 12:59' and 'No Changes'.

Status

- Changes
- Build with Parameters
- Configure
- Full Stage View
- Rename
- Pipeline Syntax

Builds

Filter

Today

#732 12:59 PM

Stage View

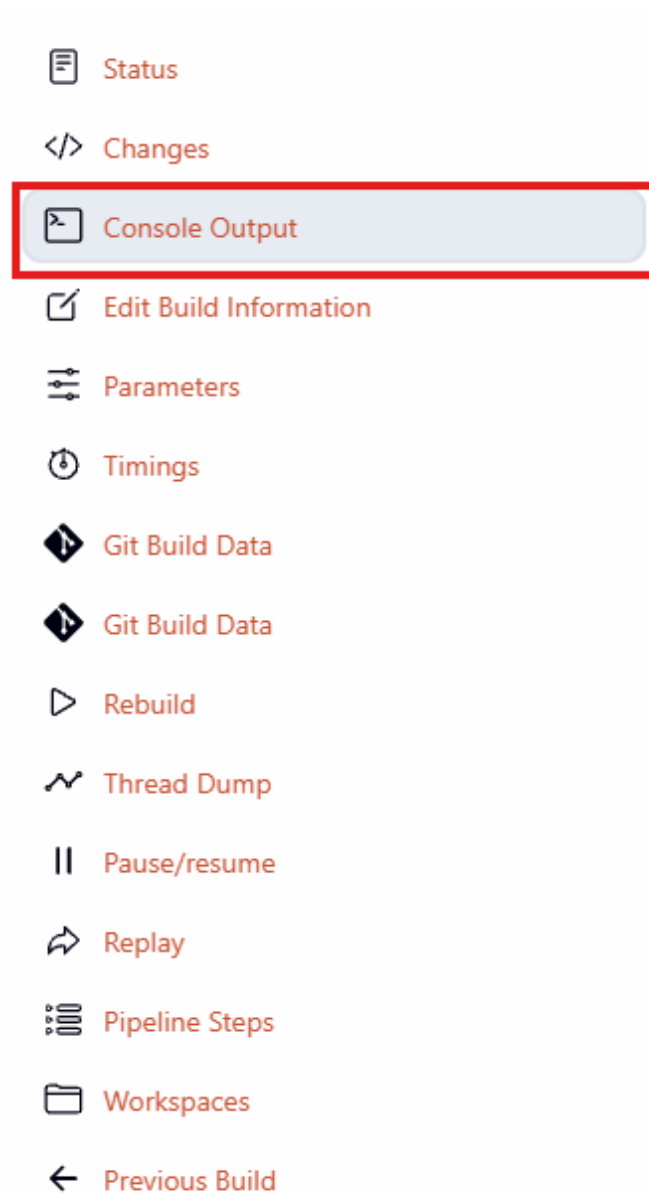
Average stage times:

#732

Sep 24 12:59 No Changes

#731

Sep 24 12:59 No Changes



In this console, you can monitor the process live. **Note that initiating a new job will abort the previous one!**

The entire process comprises several stages:

- **Declarative: Checkout SCM** – Prepare check code environment
- **Init** – Prepare Continuous Integration Flow
- **Download Source Code** – Retrieves team repository. Only “main” branch is used for solution verification.
- **Build Judge** – compilation of judge software if necessary.
- **Quality check** – single VCS compilation run over rtl.
- **Siemens simulation** – task 6 evaluation run.
- **Synopsys simulation** - task 4 evaluation run.
- **Code Synthesis** – Generates the bitstream for evaluation. This stage may take time, depending on the solution code. The timeout for this phase will be announced at the beginning of the mission.
- **Reserve Board, Program, and Judge Bitfile**- Selects one of the KRIA KV260 boards for bitfile evaluation and solution quality assessment. The bitfile will not be loaded if the timing requirement is not met.
- **Obtain results** – Updating Scoreboard website
- **Declarative: Post Actions** - Checking if any error occurred

If you encounter errors in any of the following phases, please contact the environment team:

- Declarative: Checkout SCM
- Init
- Download Source Code
- Reserve Board, Program, and Judge Bitfile
- Obtain results
- Declarative: Post Actions

You will need to debug any errors that occur in the **Code Synthesis** phase. Mentors can help, but probably it is due to user mistake.

7. Error Management Protocol (Common issues)

It's a best practice to thoroughly review logs from Jenkins and the tools utilized during the event. When examining Vivado logs, pay attention not only to fatal errors but also to those labeled as "CRITICAL WARNING." Vivado's sensitivity differs from most programs, so it's crucial to consider "CRITICAL WARNING" during troubleshooting.

1. Timing Report Generation Issue

There may be instances where Vivado's synthesis and implementation succeed, yet timing issues arise. If you encounter the following syntax in your synthesis/implementation log:

CRITICAL WARNING: [Timing 38-282] The design failed to meet the timing requirements. Please see the timing summary report for details on the timing violations.

```
INFO: [runtcl-4] Executing : report_route_status -file design_1_wrapper_route_status.rpt -pb design_1_wrapper_route_status.pb
INFO: [runtcl-4] Executing : report_timing_summary -max_paths 10 -report_unconstrained -file design_1_wrapper_timing_summary_routed.rpt -pb design_1_wrapper_timing_summary_routed.pb -rpx design_1_wrapper_timing_summary_routed.rpx -warn_on_violation
INFO: [Timing 38-91] UpdateTimingParams: Speed grade: -2LV, Temperature grade: C, Delay Type: min_max.
INFO: [Timing 38-191] Multithreading enabled for timing update using a maximum of 8 CPUs
CRITICAL WARNING: [Timing 38-282] The design failed to meet the timing requirements. Please see the timing summary report for details on the timing violations.
INFO: [runtcl-4] Executing : report_incremental_reuse -file design_1_wrapper_incremental_reuse_routed.rpt
INFO: [Vivado_Tcl 4-1062] Incremental flow is disabled. No incremental reuse info to report.
INFO: [runtcl-4] Executing : report_clock_utilization -file design_1_wrapper_clock_utilization_routed.rpt
INFO: [runtcl-4] Executing : report_bus_skew -warn_on_violation -file design_1_wrapper_bus_skew_routed.rpt -pb design_1_wrapper_bus_skew_routed.pb -rpx design_1_wrapper_bus_skew_routed.rpx
INFO: [Timing 38-91] UpdateTimingParams: Speed grade: -2LV, Temperature grade: C, Delay Type: min_max.
INFO: [Timing 38-191] Multithreading enabled for timing update using a maximum of 8 CPUs
INFO: [Timing 38-450] Writing timing data to binary archive.
Write ShapeDB Complete: Time (s): cpu = 00:00:00.15 ; elapsed = 00:00:00.03 . Memory (MB): peak = 5234.707 ; gain = 0.000 ; free physical = 148703 ; free virtual = 185233
Write PlaceDB: Time (s): cpu = 00:00:00.02 ; elapsed = 00:00:00.63 . Memory (MB): peak = 5234.707 ; gain = 0.000 ; free physical = 148686 ; free virtual = 185231
Write PulsedLatchedDB: Time (s): cpu = 00:00:00 ; elapsed = 00:00:00 . Memory (MB): peak = 5234.707 ; gain = 0.000 ; free physical = 148686 ; free virtual = 185231
Writing XDEF routing.
```

You must refine your code to resolve this issue and ensure that it does not recur in subsequent synthesis/implementation runs.

2. Out of Memory (OOM) Issue on Jenkins

If you observe the following error in your Jenkins log:

Container check-code was terminated (Exit Code: 137, Reason: OOMKilled)

```
dev/check-code-vivado-674-6w5m6-0b821-3dz68 Container check-code was terminated (Exit Code: 137, Reason: OOMKilled)

- check-code -- terminated (137)
-----Logs-----
```

Please create a ticket on Discord, including "OOM Issue" in the ticket description. This error signifies that certain processing steps (compilation, synthesis, etc.) require more than the allocated 24GiB of memory, likely due to suboptimal implementation. This issue should not occur and will be addressed individually.

8. Tool Usage Protocol (What is available for work?)

8.1. Vivado

Open terminal and type:

```
vivado
```

For more details refer to [FPGA_Hackathon_2026_Project_Guide.pdf](#)

8.2. Matlab

Open terminal and type:

```
matlab
```

8.3. Siemens Tools

Refer to [FPGA_Hackathon_2026_Project_Guide.pdf](#) for instructions on running different Siemens Tools.

8.5. VCS

Execute `quality_check` script to initiate `quality_check` steps locally.

For detailed instructions, refer to [FPGA_Hackathon_2026_Project_Guide.pdf](#)

8.6. Verdi

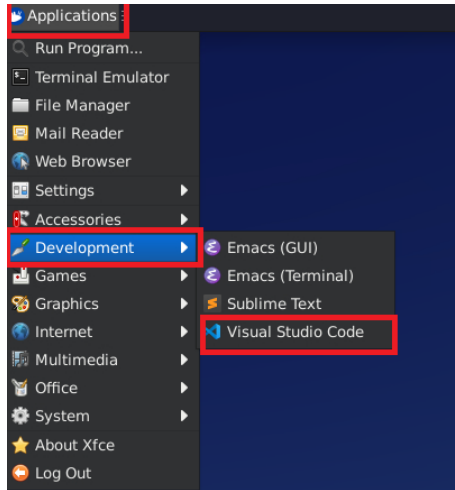
Open terminal and type:

```
verdi
```

8.7. Text Editors

8.7.1. Visual Studio Code

Navigate to the top right corner of the desktop. From the “Applications” drop-down menu, select “Visual Studio Code” under the “Development” section.



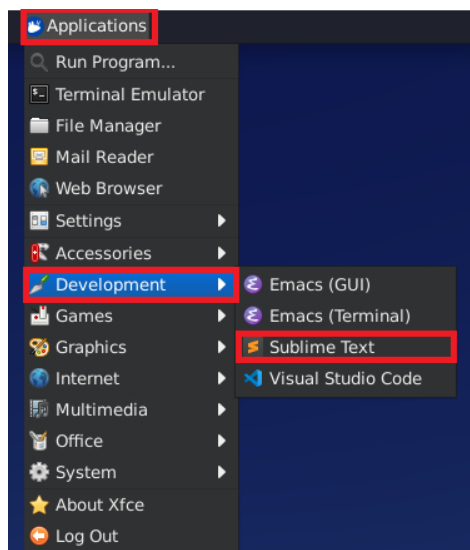
8.7.2. Gedit

Open terminal and type:

```
gedit <file>
```

8.7.3. Sublime Text

Navigate to the top right corner of the desktop. From the “Applications” drop-down menu, select “Sublime Text” under the “Development” section.



8.7.4. Emacs

Open terminal and type:

```
emacs <file>
```

8.7.5 Nano

Open terminal and type:

```
nano <file>
```

8.7.6. Vim with NERDTree extension

Open terminal and type:

```
vim <file>
```

9. Mission Conclusion

As you embark on this journey, remember to utilize the tools and protocols outlined above with precision and focus. Your expertise and dedication are crucial to the success of this mission.

Good luck, and your efforts may lead to outstanding achievements!